PYTHON AVANZADO

CLOSURES — CIERRES EN PYTHON

Guía Técnica Completa

Análisis paso a paso con diagramas de memoria.

Python permite que una función anidada acceda al ámbito externo de la función de cierre. Esto se conoce como cierre. Echemos un vistazo a cómo funcionan los cierres en Python. En Python, el cierre se crea anidando una función dentro de otra función de encapsulación y luego devolviendo la función interna. Vea el ejemplo a continuación.

```
# Código completo del ejemplo
def add_ten():
    ten = 10
    def add(num):
        return num + ten
    return add

closure_result = add_ten()
print(closure_result(5))   # → 15
print(closure_result(10))  # → 20
```

Capítulo 1: ¿Qué es un Closure en Python?

Un closure (cierre) es una función interna que recuerda y puede acceder a variables del ámbito léxico de su función externa, incluso después de que esa función externa haya terminado de ejecutarse. Este comportamiento es una de las características más poderosas y elegantes de Python.

Aclaración: El término **léxico significa** que el alcance de una variable se determina **exclusivamente por la ubicación física del código fuente** antes de que el programa se ejecute.

Durante la fase de compilación (cuando Python transforma tu texto en un *Abstract Syntax Tree* y luego en *Bytecode*), el compilador mapea exactamente qué función está anidada dentro de qué otra. La resolución de variables no depende de cómo o cuándo llamas a la función (la pila de ejecución), sino de **dónde la escribiste**.

En Python, esto se rige por la regla de resolución de nombres **LEGB**:

1. **L**ocal (El frame actual).
2. **E**nclosing (El ámbito de la **función externa**).
3. **G**lobal (El diccionario del módulo).
4. **B**uilt-in (Las primitivas del lenguaje).

Lo que preguntas es la capa **"E" (Enclosing)**.

💡 **Definición Formal**

Un closure es una función que captura y mantiene una referencia al entorno léxico (scope) en el que fue creada. Esto le permite acceder a variables libres — variables que no son locales a la función, pero tampoco son globales.

1.1 Los tres requisitos de un Closure

Para que exista un closure en Python, deben cumplirse

exactamente tres condiciones:

#	Requisito	Descripción
1	**Función anidada**	Debe existir una función definida dentro de otra función.
2	**Acceso al ámbito externo**	La función interna debe referenciar al menos una variable del ámbito externo.

3	**La función externa retorna la interna**	La función externa debe retornar la función interna como valor (no la llama, la devuelve).

Capítulo 2: Análisis del Código Completo

Antes de recorrer los diagramas paso a paso, analicemos la estructura completa del programa de ejemplo. Este código es simple pero captura perfectamente la esencia de un closure.

2.1 Código fuente completo

```
# ── FUNCIÓN EXTERNA ──────────────────────────────
def add_ten():
    ten = 10          # Variable del ámbito externo

    # ── FUNCIÓN INTERNA (cierre) ──────────────────
    def add(num):     # num es el parámetro local
        return num + ten  # 'ten' viene del ámbito externo

    return add        # Devuelve la función SIN llamarla

# ── USO DEL CLOSURE ──────────────────────────────
closure_result = add_ten()     # closure_result apunta a add()
print(closure_result(5))       # → 15  (5 + 10)
print(closure_result(10))      # → 20  (10 + 10)
```

2.2 Anatomía del código

Elemento	Explicación
`add_ten()`	Función externa (enclosing function). Define el contexto que será capturado.
`ten = 10`	Variable libre. Pertenece al ámbito de add_ten, pero será capturada por add.
`def add(num)`	Función interna (closure). Esta es la función que se convierte en closure.
`return num + ten`	La función add accede a 'ten' del ámbito externo. Esta es la magia del closure.
`return add`	Se retorna la función add SIN llamarla. Se devuelve el objeto función.
`closure_result`	Variable global que almacena la referencia al closure add, junto con su entorno capturado.

Capítulo 3: Ejecución Paso a Paso con Diagramas de Memoria

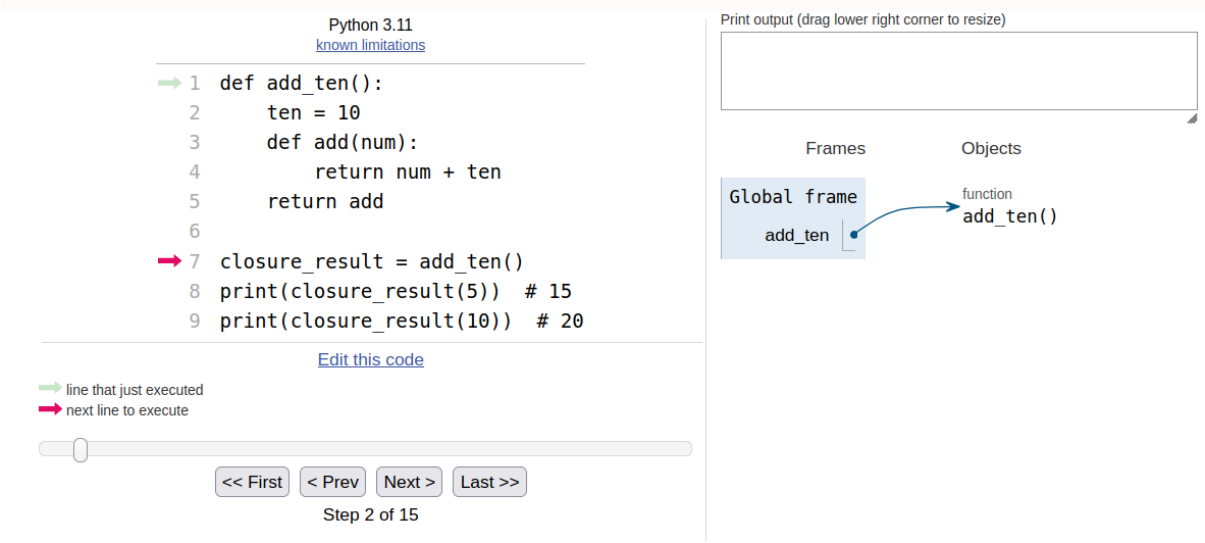
Ahora recorreremos cada uno de los 15 pasos de ejecución del programa, tal como los muestra Python Tutor. Los diagramas muestran el estado exacto de la memoria (frames y objetos) en cada instante.

¿Qué es Python Tutor?
Python Tutor es una herramienta educativa que visualiza la ejecución de código Python paso a paso, mostrando el estado de los marcos de pila (frames) y objetos en memoria. Usaremos sus diagramas como referencia.

3.1 Paso 1 — Inicio del programa

Paso 1/15 **Python carga el módulo — la flecha apunta a la línea 1**

En el primer paso, Python empieza a interpretar el archivo. El cursor (flecha roja) señala la línea 1: def add_ten():. Aún no se ejecuta nada; solo se prepara el entorno de ejecución.



3.3 Paso 3 — Se llama a add_ten(), se crea su frame

Paso 3/15 **add_ten() es invocada — se crea el frame f1: add_ten**

Se ejecuta la instrucción closure_result = add_ten(). Esto invoca la función add_ten, lo que provoca la creación de un nuevo marco de pila (stack frame) denominado f1: add_ten. En este momento, el frame global y f1 coexisten en memoria.

La etiqueta **`f1`** es simplemente un identificador secuencial inventado por el visualizador que significa **"Frame 1"**.

Sin embargo, a nivel de la máquina, representa algo muy concreto: una **instancia de la estructura C llamada `PyFrameObject`**.

Cuando el intérprete lee la instrucción de llamar a `add_ten()` en la línea 7, necesita un espacio de memoria aislado para guardar las variables locales que esa función va a usar (en este caso, la variable `ten` y la definición de la sub-función `add`). CPython solicita al sistema operativo un bloque de memoria RAM para instanciar este `PyFrameObject`. El visualizador lo bautiza `f1` para que puedas rastrearlo.

Esta es la descripción de la mecánica fundamental del **Call Stack (Pila de Llamadas)** del procesador.

El código fuente de Python se ejecuta en un solo hilo (single-threaded por defecto). Esto significa que el procesador solo puede leer y ejecutar una instrucción a la vez.

Cuando llegas a la línea 7 (`closure_result = add_ten()`), el flujo de control se enfrenta a un problema: el entorno Global está a mitad de hacer una asignación, pero necesita el resultado de `add_ten()` para terminarla. La máquina resuelve esto así:

1. **El Global Frame se "congela":** El bloque de memoria del Global Frame (que contiene las variables globales iniciales) **no se borra**. Su estado actual se guarda y el puntero de instrucción del entorno Global se pausa temporalmente en la línea 7.
2. **Se "empuja" f1 a la pila (Push):** El nuevo bloque de memoria instanciado (`f1`) se coloca literalmente "encima" del Global frame en la estructura de datos del Call Stack.
3. **Cambio de contexto:** El intérprete mueve su "lupa" (el puntero de ejecución activo) al código interno de `add_ten()` (línea 2) dentro del espacio de memoria de `f1`.

La coexistencia significa que ambos bloques ocupan RAM al mismo tiempo. El Global Frame está en la base de la pila reteniendo sus referencias globales, suspendido y esperando. El Frame `f1` está en la cima de la pila, siendo procesado activamente.

Esta coexistencia es imperativa: si el Global Frame se borrara de la memoria para hacerle espacio a `f1`, cuando `f1` terminara y devolviera su valor, el programa no tendría a dónde regresar el resultado, porque quien lo llamó (el Global) ya no existiría.

```
1   def add_ten():
2       ten = 10
3       def add(num):
4           return num + ten
```

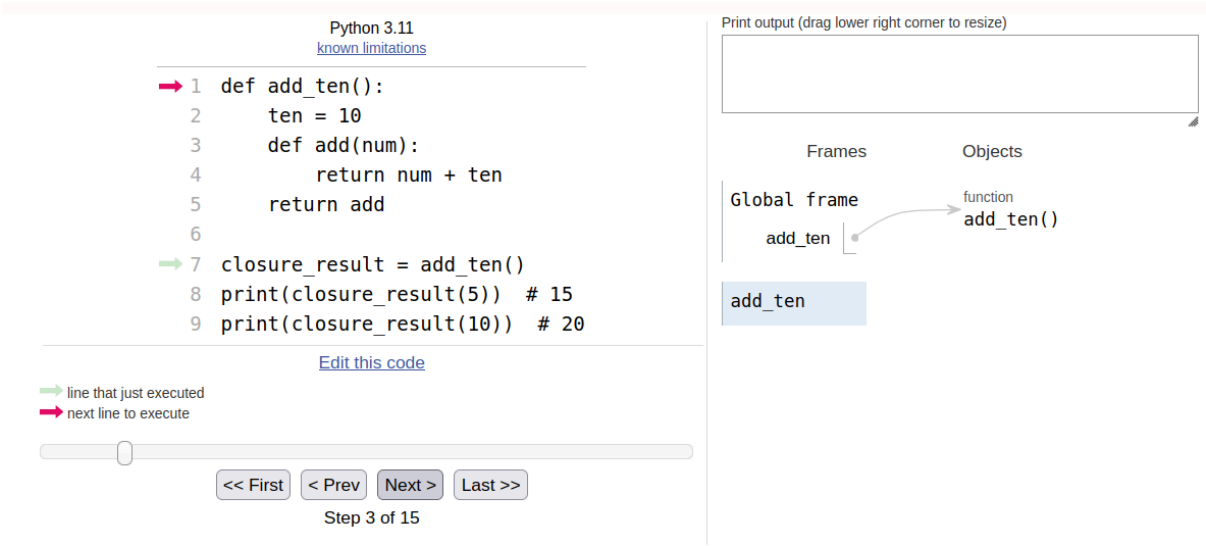
Estado de la Memoria

Global frame: frame	
add_ten	→ function add_ten()

```
5        return add
6
7 → closure_result = add_ten()
8   print(closure_result(5))    # 15
9   print(closure_result(10))   # 20
```

add_ten: f1

→ *Se creó el frame f1 para ejecutar add_ten.*



3.4 Paso 4 — Se asigna ten = 10 en f1

Paso 4/15 **La variable 'ten' se crea en el frame f1 con valor 10**

Dentro del frame f1 (el de add_ten), Python ejecuta la asignación ten = 10. La variable ten existe únicamente dentro de f1; es una variable local de add_ten. En este momento, ten almacena el entero 10.

```
1   def add_ten():
2 →     ten = 10
3       def add(num):
4           return num + ten
5       return add
```

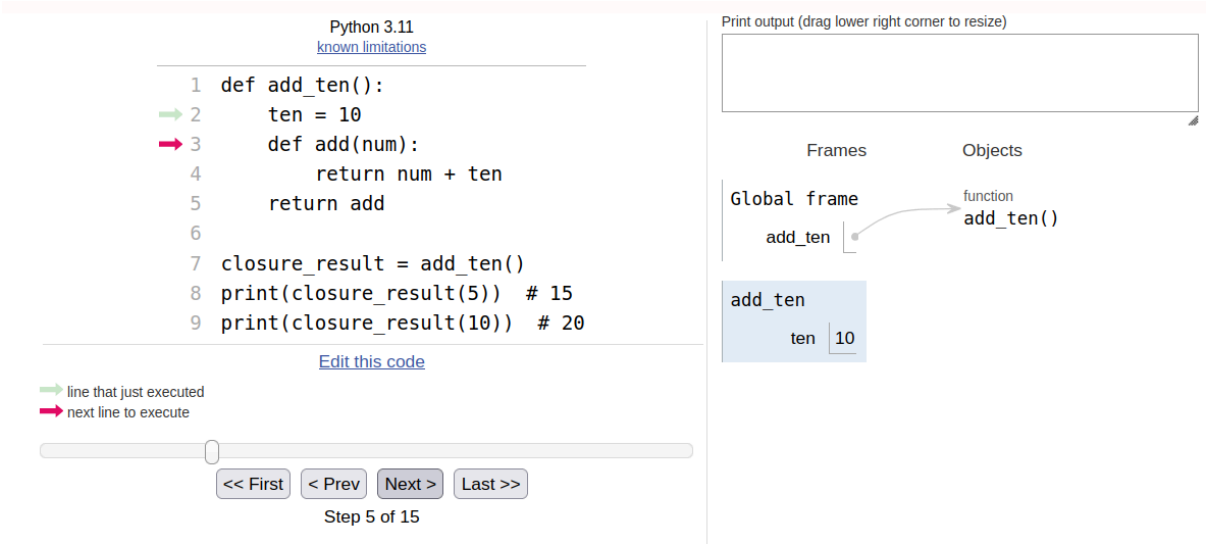
Estado de la Memoria

Global frame: frame	
add_ten	→ function add_ten()

```
6
7   closure_result = add_ten()
8   print(closure_result(5))    # 15
9   print(closure_result(10))   # 20
```

add_ten: f1	
ten	10

→ ten = 10 vive en f1. Es la variable que el closure capturará



3.5 Paso 5 — Se define add() dentro de add_ten

Paso 5/15 **def add(num) crea el objeto función interno con parent=f1**

Python procesa la instrucción def add(num):. Igual que antes, no ejecuta el cuerpo de add; crea un objeto función en memoria. La diferencia crucial es que este objeto función guarda una referencia a su frame padre (parent=f1), capturando así el entorno de add_ten.

> 💡 **¡Aquí nace el closure!**
> Cuando Python crea el objeto función add, automáticamente le adjunta una referencia al frame f1 (que contiene ten = 10). Esta referencia es lo que convierte a add en un closure: lleva consigo su entorno.

```
1   def add_ten():
2       ten = 10
3 →     def add(num):
```

Estado de la Memoria

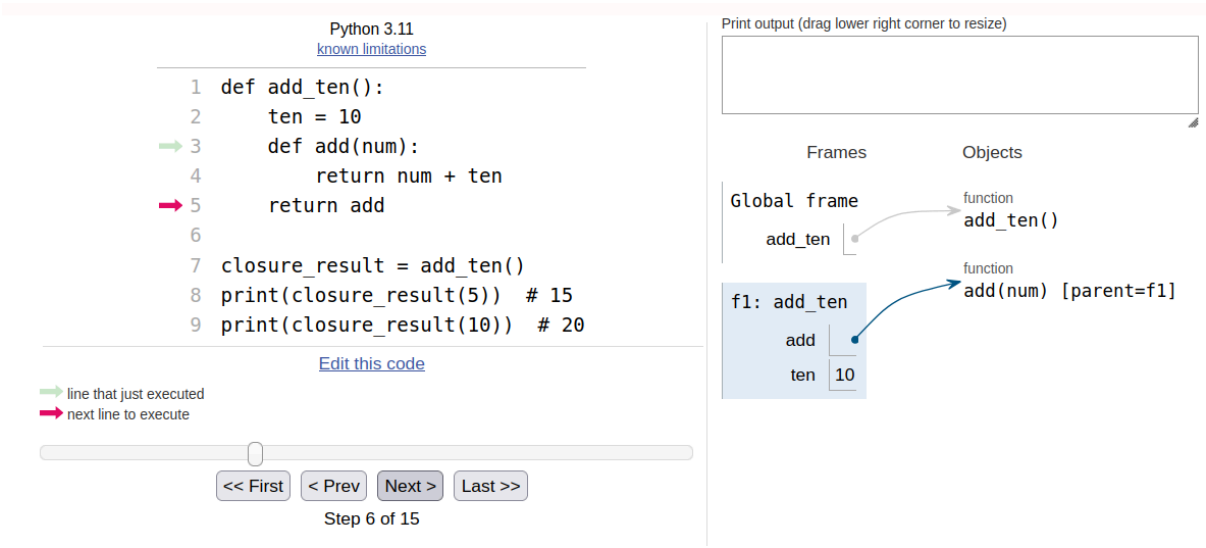
Global frame: frame

```
4            return num + ten
5        return add
6
7    closure_result = add_ten()
8    print(closure_result(5))    # 15
9    print(closure_result(10))   # 20
```

add_ten	→ function add_ten()

add_ten: f1	
ten	10
add	→ function add(num) [parent=f1]

→ add(num) tiene [parent=f1]: captura el entorno de add_ten.



3.6 Paso 6 — add_ten retorna add

Paso 6/15 **return add — la función interna es devuelta como valor**

Se ejecuta return add. Esto no llama a add sino que devuelve el objeto función add (incluyendo su referencia a f1) como valor de retorno de add_ten. El frame f1 queda en un estado especial: aunque add_ten haya terminado, f1 no puede ser destruido porque add aún mantiene una referencia a él.

```
1    def add_ten():
2        ten = 10
3        def add(num):
4            return num + ten
```

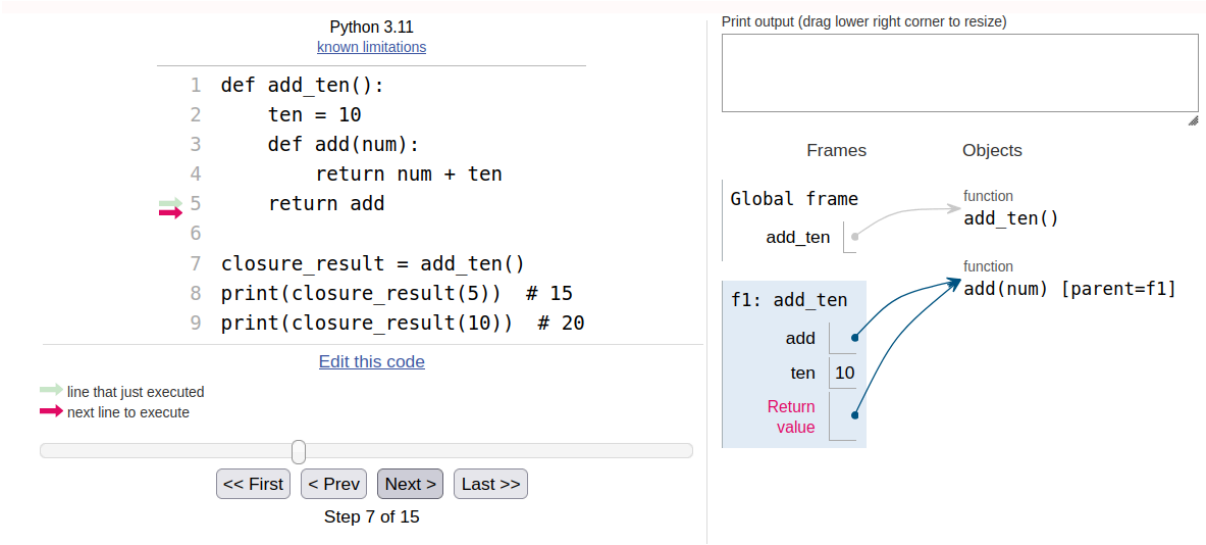
Estado de la Memoria

Global frame: frame	
add_ten	→ function add_ten()

```
5 →     return add
6
7   closure_result = add_ten()
8   print(closure_result(5))    # 15
9   print(closure_result(10))   # 20
```

add_ten: f1	
add	→ function add(num) [parent=f1]
ten	10
Return value	→ function add(num) [parent=f1]

→ f1 persiste en memoria gracias al closure.



3.7 Paso 7 — closure_result recibe la función add

Paso 7/15 **closure_result = add_ten() completa la asignación**

La asignación closure_result = add_ten() se completa. closure_result en el frame global ahora apunta al mismo objeto función add que antes fue retornado por add_ten. Crucialmente, ese objeto función aún tiene su referencia a f1 (donde ten = 10).

```
1   def add_ten():
2       ten = 10
3       def add(num):
4           return num + ten
```

Estado de la Memoria

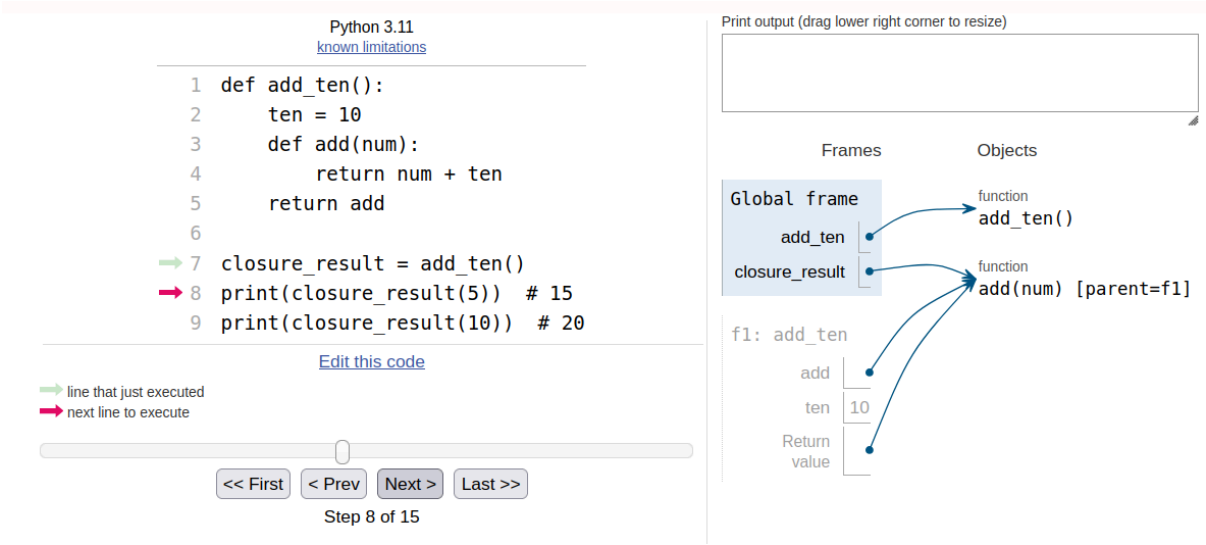
Global frame: frame	
add_ten	→ function add_ten()

```
5        return add
6
7 → closure_result = add_ten()
8   print(closure_result(5))    # 15
9   print(closure_result(10))   # 20
```

closure_result	→ function add(num) [parent=f1]

add_ten: f1	
add	→ add(num)
ten	10
Return value	→ add

→ closure_result es ahora un alias del closure add.



3.8 Pasos 8-9 — Primera llamada: closure_result(5)

Paso 8/15 **closure_result(5) invoca add con num=5**

Python evalúa print(closure_result(5)). Primero debe resolver closure_result(5). Esto invoca la función add con num=5, creando un nuevo frame para add. Dentro de ese frame, num=5 es una variable local, y ten=10 se accede desde el frame padre f1.

```
1   def add_ten():
2       ten = 10
3       def add(num):
4 →         return num + ten
5       return add
```

Estado de la Memoria

Global frame: frame	
add_ten	→ function add_ten()

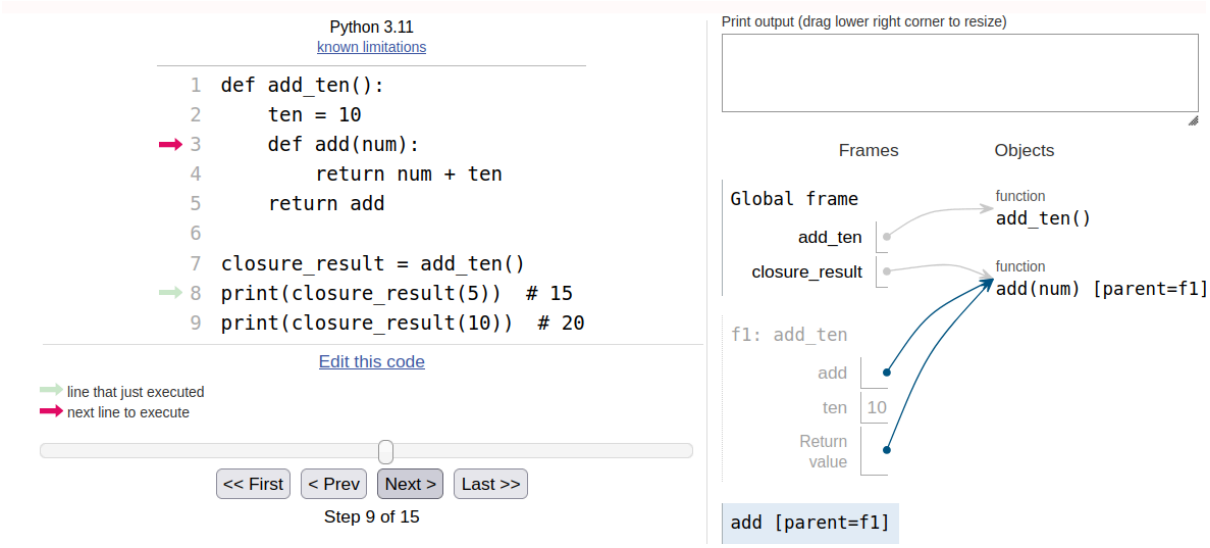
```
6
7   closure_result = add_ten()
8   print(closure_result(5))    # 15
9   print(closure_result(10))   # 20
```

closure_result	→ add(num) [parent=f1]

add_ten: f1	
add	→ add(num)
ten	10

frame local: add [parent=f1]	
num	5
Return value	15

→ add ve ten=10 a través del parent=f1. Retorna 5+10=15.



Aquí es precisamente donde el algoritmo **LEGB** (Local, Enclosing, Global, Built-in) entra en acción a nivel de arquitectura, y funciona estrictamente porque Python utiliza un **Ámbito Léxico (Lexical Scoping)**.

No hay "magia" ni "recuerdos abstractos". Hay una estructura de datos muy específica en el lenguaje C subyacente que permite este comportamiento. Aquí tienes la explicación a nivel de diseño de compiladores:

El Algoritmo LEGB en el Procesador

Cuando el puntero de instrucción llega a la línea 4: `return num + ten`, CPython debe traducir los identificadores de texto (`num` y `ten`) en direcciones de memoria reales. Aplica el algoritmo LEGB estrictamente en orden:

- **Para el identificador `num`:**
  - **L (Local):** El intérprete busca en el diccionario del frame activo (`frame local: add`). Encuentra la clave `num`. Su valor es un puntero al objeto entero `5`. Búsqueda terminada.
- **Para el identificador `ten`:**
  - **L (Local):** El intérprete busca en el frame activo (`add`). No existe ninguna clave llamada `ten`. Falla.
  - **E (Enclosing / Ámbito Contenedor):** El intérprete sube un nivel en la jerarquía léxica. Busca en el entorno de la función que *envolvió* a esta función durante el momento de su compilación (el frame `f1`). Encuentra la clave `ten`. Su valor es un puntero al objeto entero `10`. Búsqueda terminada.

2. ¿Cómo "recuerda" el ámbito léxico si la función original ya terminó?

Aquí está el núcleo técnico de tu pregunta. En el Paso 7 (`closure_result = add_ten()`), la función `add_ten` terminó su ejecución. Según las reglas estándar de la pila de llamadas (Call Stack), el frame `f1` debería haber sido destruido y la variable `ten` eliminada de la memoria.

¿Por qué el visualizador te muestra `[parent=f1]` y cómo sobrevive `ten`? Esto se logra mediante un mecanismo arquitectónico llamado **Closure (Cierre o Clausura)**. CPython no lo hace por accidente, lo planifica desde que compila el código por primera vez:

1. **Detección de Variables Libres (Free Variables):** Cuando CPython compila la función interna `add(num)` (líneas 3-4), el compilador se da cuenta de que la variable `ten` no es local ni global. La marca internamente como una **variable libre**.
2. **Creación de PyCellObjects:** Debido a que `ten` es una variable libre, el intérprete cambia la forma en que la almacena en memoria. En lugar de poner `ten` como una simple variable local efímera dentro del frame `f1`, CPython crea un objeto especial en el Heap llamado **`PyCellObject`** (un objeto Celda).
3. **El Enganche (The Binding):** * El frame `f1` apunta a este objeto celda.
   - El objeto función `add` recién instanciado también recibe un puntero hacia este mismo objeto celda. Este puntero se guarda en un atributo oculto de la función llamado `__closure__`.
4. **Supervivencia:** Cuando `add_ten` termina en la línea 7 y su frame en la pila se destruye, el contador de referencias (Reference Count) del objeto celda no baja a cero. ¿Por qué? Porque la función `add` (que ahora vive en la variable global `closure_result`) **sigue manteniendo un puntero hacia esa celda en el Heap**.

En Resumen (Nivel Libro de Arquitectura)

El visualizador de Python Tutor dibuja un "fantasma" de f1 etiquetado como [parent=f1] para que los humanos lo entiendan fácilmente.

Pero en la RAM de tu computadora en este Paso 8-9, lo que está pasando es que la función invocada (closure_result) tiene un atributo a nivel de C (__closure__) que es una tupla de punteros. El algoritmo LEGB, al fallar en la 'L', pasa a la 'E', lo que significa que el intérprete simplemente va a leer esa tupla __closure__, extrae el puntero del PyCellObject correspondiente, y accede a la dirección de memoria donde reside el objeto entero 10.

Eso es el Ámbito Léxico: la capacidad estructural del objeto función de llevar consigo (como una mochila de punteros) las referencias de memoria de su entorno original, sin importar desde qué parte del código global sea invocado posteriormente.

3.9 Pasos 10-12 — El print muestra 15

Paso 10/15 **print recibe 15 como argumento — lo muestra en pantalla**

El valor de retorno 15 se pasa a print(), que lo muestra en la consola. El frame temporal de add se descarta, pero f1 permanece porque closure_result aún mantiene viva la referencia al closure.

```
1   def add_ten():
2       ten = 10
3       def add(num):
4           return num + ten
5       return add
6
7   closure_result = add_ten()
8 → print(closure_result(5))    # 15
9   print(closure_result(10))   # 20
```

Salida en Consola

```
15
```

→ *El closure calculó 5 + 10 = 15 correctamente.*

Python 3.11
known limitations

```
1 def add_ten():
2     ten = 10
3     def add(num):
4         return num + ten
5     return add
6
7 closure_result = add_ten()
8 print(closure_result(5))  # 15
9 print(closure_result(10))  # 20
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 10 of 15

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
add_ten → function add_ten()
closure_result → function add(num) [parent=f1]

f1: add_ten
add
ten 10
Return value

add [parent=f1]
num 5

Python 3.11
known limitations

```
1 def add_ten():
2     ten = 10
3     def add(num):
4         return num + ten
5     return add
6
7 closure_result = add_ten()
8 print(closure_result(5))  # 15
9 print(closure_result(10))  # 20
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 11 of 15

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

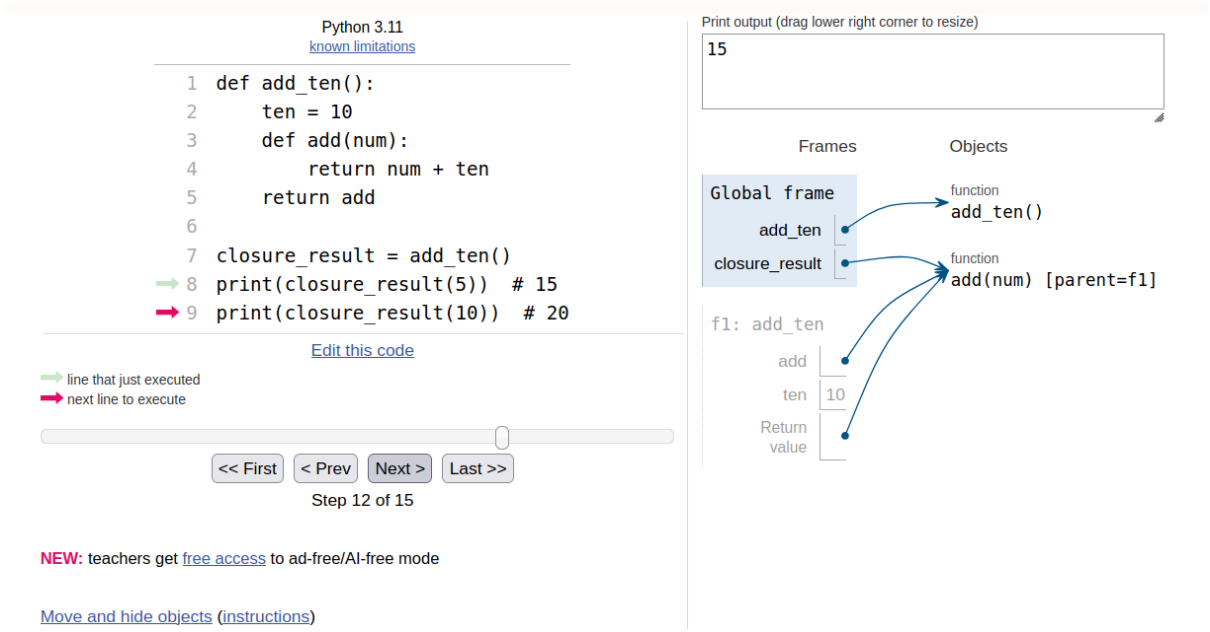
Print output (drag lower right corner to resize)

Frames | Objects

Global frame
add_ten → function add_ten()
closure_result → function add(num) [parent=f1]

f1: add_ten
add
ten 10
Return value

add [parent=f1]
num 5
Return value 15



3.10 Pasos 13-15 — Segunda llamada: closure_result(10)

Paso 13/15 **closure_result(10) invoca add nuevamente con num=10**

Se ejecuta print(closure_result(10)). El mismo closure es invocado de nuevo, esta vez con num=10. El entorno capturado (ten=10) no ha cambiado; sigue siendo el mismo f1. Python crea un nuevo frame local para esta segunda invocación de add.

> 💡 **Observación clave**
> El closure puede ser invocado múltiples veces. Cada llamada crea un frame local nuevo para los parámetros y variables locales, pero todos comparten el mismo entorno capturado (f1 con ten=10).

```
1   def add_ten():
2       ten = 10
3       def add(num):
4 →         return num + ten
5       return add
6
7   closure_result = add_ten()
8   print(closure_result(5))    # 15
9   print(closure_result(10))   # 20
```

Estado de la Memoria

Global frame: frame	
add_ten	→ add_ten()
closure_result	→ add(num) [parent=f1]

add_ten: f1	
add	→ add(num)
ten	10

frame local: add [parent=f1]	
num	10
Return value	20

→ add ve ten=10 de f1. Retorna 10+10=20.

Python 3.11
known limitations

```
1  def add_ten():
2      ten = 10
3      def add(num):
4          return num + ten
5      return add
6
7  closure_result = add_ten()
8  print(closure_result(5))  # 15
9  print(closure_result(10))  # 20
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 13 of 15

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

```
15
```

Frames | Objects

Global frame
add_ten → function add_ten()
closure_result → function add(num) [parent=f1]

f1: add_ten
add
ten 10
Return value

add [parent=f1]
num 10

Python 3.11
known limitations

```
1  def add_ten():
2      ten = 10
3      def add(num):
4          return num + ten
5      return add
6
7  closure_result = add_ten()
8  print(closure_result(5))  # 15
9  print(closure_result(10))  # 20
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 14 of 15

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

```
15
```

Frames | Objects

Global frame
add_ten → function add_ten()
closure_result → function add(num) [parent=f1]

f1: add_ten
add
ten 10
Return value

add [parent=f1]
num 10

Python 3.11
known limitations

```
1 def add_ten():
2     ten = 10
3     def add(num):
4         return num + ten
5     return add
6
7 closure_result = add_ten()
8 print(closure_result(5))  # 15
9 print(closure_result(10))  # 20
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 15 of 15

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

```
15
```

Frames | Objects

Global frame
add_ten → function add_ten()
closure_result → function add(num) [parent=f1]

f1: add_ten
add
ten 10
Return value

add [parent=f1]
num 10
Return value 20

Python 3.11
known limitations

```
1 def add_ten():
2     ten = 10
3     def add(num):
4         return num + ten
5     return add
6
7 closure_result = add_ten()
8 print(closure_result(5))  # 15
9 print(closure_result(10))  # 20
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (15 steps)

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

```
15
20
```

Frames | Objects

Global frame
add_ten → function add_ten()
closure_result → function add(num) [parent=f1]

f1: add_ten
add
ten 10
Return value

3.11 Resultado Final

Paso 15/15 **Done running (15 steps) — programa completado**

El programa termina. La salida en consola muestra los dos resultados calculados por el closure. El estado final de la memoria mantiene el frame global con closure_result apuntando al closure add (con su parent f1 aún vivo).

```
# Salida en consola:
15
20
```

Capítulo 4: Conceptos Avanzados de Closures

4.1 La regla LEGB y los closures

Python busca nombres de variables siguiendo la regla LEGB (Local → Enclosing → Global → Built-in). Los closures se apoyan exactamente en el nivel E (Enclosing — de cierre).

L	**Local**	Variables definidas dentro de la función actual.
E	**Enclosing**	Variables del ámbito de la función que contiene el closure. ← AQUÍ ACTÚAN LOS CLOSURES
G	**Global**	Variables del módulo o script principal.
B	**Built-in**	Nombres predefinidos de Python como print, len, range, etc.

4.2 Inspeccionar un closure con __closure__

Python expone los detalles de un closure a través de atributos especiales. Puedes inspeccionarlos para entender qué variables ha capturado.

```
closure_result = add_ten()

# Ver las celdas capturadas
print(closure_result.__closure__)        # (<cell at 0x...>,)
print(closure_result.__closure__[0].cell_contents)  # 10

# Ver los nombres de las variables libres
print(closure_result.__code__.co_freevars)   # ('ten',)
```

4.3 Uso con nonlocal — modificar variables capturadas

Por defecto, un closure solo puede leer las variables del ámbito externo, no reasignarlas. Para modificarlas, debes usar la palabra clave nonlocal.

```
def make_counter():
    count = 0
```

```
    def increment():
        nonlocal count    # Permite modificar 'count'
        count += 1
        return count
    return increment

counter = make_counter()
print(counter())  # 1
print(counter())  # 2
print(counter())  # 3
```

4.4 Aplicaciones prácticas de los closures

Caso de uso	Descripción
Fábricas de funciones	Crear funciones especializadas a partir de una función genérica (como add_ten).
Decoradores	Los decoradores en Python son esencialmente closures que envuelven otra función.
Encapsulación de estado	Mantener estado privado sin necesidad de clases (alternativa ligera a OOP).
Callbacks y event handlers	Capturar contexto local para usarlo más tarde en un callback asíncrono.
Memoización / caché	Guardar resultados de llamadas previas en el entorno capturado.

Capítulo 5: Resumen Visual del Proceso

El siguiente diagrama muestra el flujo completo de vida de un closure en Python, desde la definición hasta las múltiples invocaciones.

1. `def add_ten():` →	Se registra add_ten en el frame global.
2. `add_ten() se invoca` →	Se crea el frame f1. ten = 10 vive aquí.
3. `def add(num): [parent=f1]` →	Se crea el closure add, apuntando a f1.
4. `return add` →	add_ten devuelve el objeto función add (con su entorno f1).
5. `closure_result = add_ten()` →	closure_result ahora apunta al closure add + f1.
6. `closure_result(5)` →	Nuevo frame local: num=5. ten=10 viene de f1. Devuelve 15.
7. `closure_result(10)` →	Nuevo frame local: num=10. ten=10 viene de f1. Devuelve 20.

💡 **Conclusión**

Un closure permite que una función interna 'recuerde' el entorno en el que fue creada, incluso cuando la función externa ya terminó. Esto se logra gracias a que Python mantiene vivo el frame externo mientras exista alguna referencia al closure.